

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
DEPARTAMENTO DE INFORMÁTICA E MATEMATICA APLICADA
Programação Distribuída

Descrição do Terceiro Trabalho

Objetivo

Desenvolver uma “AI-Powered WEB application” que faça uso de modelos de inteligência artificial como parte das suas principais funcionalidades de negócio.

Requisitos

1. Deve-se implementar uma aplicação em Java que utilize o Spring Framework.
2. Deve-se seguir o modelo arquitetural de Microserviços
3. Todos os princípios do Twelve-Factor Manifesto devem ser seguidos.
4. Desenvolvimento Individual
    - Cada aluno(a) deve implementar o seu trabalho de forma individual.
    - É expressamente proibido o uso ou a cópia de soluções desenvolvidas por outros(as) alunos(as) em semestres anteriores.
    - É expressamente proibido o uso de modelos de linguagem (LLMs), como ChatGPT, Copilot, ou similares, para a geração parcial ou total do código da aplicação.
    - Em qualquer um dos casos citados acima, caso seja identificada a infração, o(a) aluno(a) receberá nota zero neste trabalho.
5. Uso de Padrões de Projeto
    - O sistema deve ser construído seguindo os Padrões de Arquitetura de Microserviços descritos no livro "Microservices Patterns: With Examples in Java" (2019), de Chris Richardson.
6. Componentes do Sistema
    - O sistema distribuído deve conter pelo menos três microserviços, cada um responsável por uma parte da lógica de negócio. Um dos microserviços deve ser responsável pela interação com o modelo de Inteligência Artificial, enquanto um outro deve ser implementado utilizando o conceito de função Serverless.
7. O Sistema distribuído proposto deve ser resiliente a falhas de rede/componentes por meio do uso dos padrões de resiliência ensinados na terceira unidade.

8. O sistema distribuído proposto deve oferecer suporte à observabilidade, permitindo a identificação de possíveis pontos de falha, gargalos de desempenho ou situações de alta latência.

9. Protocolos de Comunicação Suportados

    - Transporte: TCP.
    - Aplicação: HTTP.

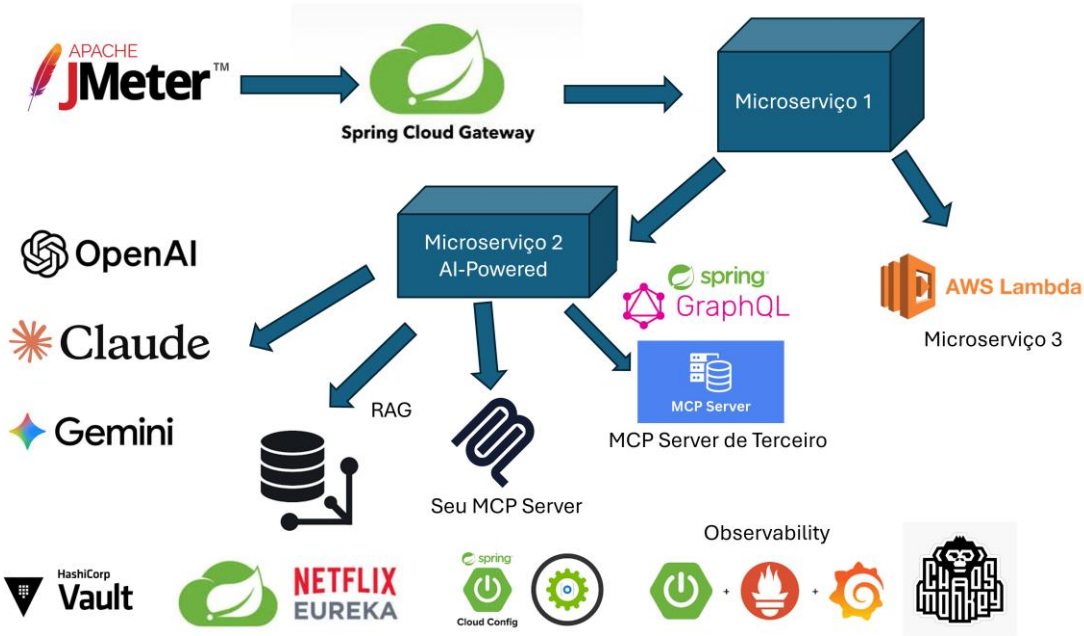
10. A aplicação deve ser disponibilizada para acesso no ambiente da AWS Academy, incluindo as instâncias AWS EC2 e outros serviços necessários.

11. Testes de Carga com JMeter

    - O aluno deve utilizar a ferramenta JMeter para realizar testes de carga do sistema.
    - No dia da apresentação do projeto, todos os testes devem estar preparados e configurados antecipadamente.

12. Deve-se investigar a capacidade do sistema, e conhecer seu **Knee capacity** e **Usable Capacity**

Abaixo, segue uma figura hipotética mostrando uma configuração mínima do sistema distribuído a ser apresentado. **Note que sua arquitetura não precisa ser exatamente como a descrita abaixo, mas precisa ter o conjunto mínimo de componentes conforme descrito abaixo se relacionando em outra configuração, por exemplo.**



Fluxo de Execução da Arquitetura

Nesta arquitetura, o **Apache JMeter** é utilizado como ferramenta de teste para gerar requisições HTTP que são encaminhadas ao **API Gateway**, implementado com **Spring Cloud Gateway**. O API Gateway atua como um **proxy reverso**, sendo responsável por receber as requisições dos clientes e roteá-las adequadamente para os microserviços internos.

Para permitir o roteamento dinâmico e escalável dos serviços, o Gateway depende de um mecanismo de **descoberta de serviços**, viabilizado por meio do **Netflix Eureka Server**, que realiza o registro e localização automática dos microserviços ativos no sistema.

A **configuração centralizada** é gerenciada pelo **Spring Cloud Config Server**, que armazena propriedades e parâmetros de ambiente de forma versionada, possibilitando alterações em tempo de execução e promovendo maior flexibilidade e consistência entre os serviços.

O sistema é composto por **três microserviços principais**, cada um com uma função específica dentro da lógica de negócio:

- **Microserviço 2**: Responsável pela **integração com modelos de Inteligência Artificial**, mais especificamente com a **API da OpenAI/Gemini/Claude/etc**. Ele atua como intermediário entre os dados da aplicação e os modelos LLM, realizando o envio de prompts e recebendo as respostas processadas. Esse serviço encapsula a lógica relacionada à IA, garantindo desacoplamento e reusabilidade. Por meio desse serviço, é possível invocar o MCP Server criado pelo aluno e o MCP Server de Terceiro, além da implementação do padrão RAG.

- **Microserviço 3**: Implementado como uma **função Serverless**, este serviço é invocado sob demanda e executa operações pontuais ou de curta duração, como transformações de dados, validações específicas ou tarefas event-driven. A abordagem Serverless garante escalabilidade automática, economia de recursos e maior eficiência para cargas intermitentes.

- **Microserviço 1**: Atua como coordenador de execução, responsável por interações com sistemas legados ou controle de processos específicos da infraestrutura da aplicação. Esse serviço centraliza a comunicação com o Microserviço 2 que deve ser feita via GraphQL. Ele garante que as requisições externas vinda do API Gateway sejam traduzidas e gerenciadas adequadamente. Neste serviço devem ser implantados os padrões de resiliência.

Adicionalmente, a arquitetura contempla **recursos de observabilidade**, essenciais para sistemas distribuídos. Isso inclui o uso de ferramentas de **monitoramento, rastreamento distribuído e análise de logs**, que permitem identificar falhas, detectar gargalos de desempenho e acompanhar a latência ao longo do fluxo de requisições. Esses mecanismos

garantem maior confiabilidade operacional e facilitam ações proativas de manutenção e otimização do sistema.

Critérios de Avaliação:

1. **Implementação dos princípios de 12 Fatores (Vale 0,5)**
2. **Uso adequado do API Gateway, Config Server, Eureka Netflix, Vault, HTTP Interface, GraphQL, Serverless Function (Vale 4,5)**
3. **Uso adequado dos padrões de Resiliência com Observabilidade (Vale 3,0)**
4. **Uso adequado do Spring AI(Chat Memory, RAG, MCPs) (Vale 2,0)**

Fluxo de execução da aplicação.

1. No **JMeter**, deve-se configurar um número de usuários simultâneos **maior que 5 e menor que o Knee Capacity**.
2. Quando a aplicação estiver em funcionamento, o **Summary Report** deve indicar **zero erros**.
3. Durante a apresentação, será solicitado o início da operação do **Chaos Monkey** para inserção de defeitos na aplicação, incluindo o desligamento de algumas instâncias. Nesse momento, espera-se que a **taxa de erro aumente**.
4. Em seguida, será solicitado que a operação do serviço seja normalizada, via desligamento do **Chaos Monkey e restauração das instancias interrompidas**. Quando isso ocorrer, a **taxa de erro deve diminuir**, demonstrando a recuperação do sistema.
5. Durante todos os passos acima, deve-se fornecer, por meio de ferramenta de observabilidade, o status das principais condições do sistema, como por exemplo, o status dos circuit breakers. A não implementação dessa observabilidade irá impactar diretamente na nota do critério 3.

Apresentação e avaliação do trabalho

As apresentações deste trabalho serão realizadas no período definido no SIGAA. A ordem das apresentações será estabelecida por sorteio.

O(a) aluno(a) deverá estar devidamente preparado(a) para iniciar sua apresentação no momento em que for chamado(a). Caso não esteja disponível ou preparado(a), perderá sua vez, podendo realizar a apresentação somente ao final da sequência, caso haja tempo disponível. Na ausência de tempo hábil para a apresentação, será atribuída nota zero ao trabalho.

Durante a apresentação, poderão ser realizadas perguntas e solicitadas demonstrações práticas dos seguintes aspectos da implementação:

1. Como os 12 fatores (The Twelve-Factor App) são suportados pela sua aplicação? (0,5 ponto)

2. Apresentação do Dashboard do Eureka Netflix, demonstrando a operação no modo Cluster. (0,5 ponto)

3. Apresentação do tipo de configuração(estática ou dinâmica) do API Gateway, incluindo os respectivos filtros e rotas utilizadas. (0,5 ponto)

4. Apresentação da configuração centralizada por meio do Config Server, utilizando um repositório Git, com demonstração da alteração dinâmica de valores de configuração. (1,0 ponto)

5. Demonstração da interface GraphQL, incluindo sua utilização na invocação de serviços. (0,5 ponto)

6. Apresentação da implementação do HTTP Interface nos serviços desenvolvidos. (0,5 ponto) e rastreamento via o *Distributed Tracing*.

7. Demonstração do funcionamento do Lambda Server. (1,0 ponto)

8. Demonstração da utilização do HashiCorp Vault. (0,5 ponto)

9. Demonstração do funcionamento do Chaos Monkey. (0,5 ponto)

10. Demonstração dos mecanismos de Retry e Fallback. (0,5 ponto)

11. Demonstração do funcionamento do Circuit Breaker. (1,0 ponto)

12. Demonstração dos mecanismos de Rate Limiter (no lado cliente e no lado servidor) e Bulkhead. (1,0 ponto)

13. Demonstração da integração da aplicação com uma LLM via Spring AI, incluindo a utilização de Chat Memory e RAG (Retrieval-Augmented Generation). (1,0 ponto)

14. Demonstração da integração com Spring AI e MCP, incluindo a implementação de um servidor MCP próprio e a utilização de um servidor MCP de terceiros. (1,0 ponto)